

参数重要性估计核心代码分析报告

0. 摘要

本系统把“参数重要性”定义在局部梯度空间。每个训练步，用同一全局 batch 的 microbatch 梯度估计 $\eta_{g(k),t}\mu_{k,t}^2$ （固定状态下的局部梯度平方目标），再跨步累计成 signed / positive / negative-mass / absolute 四个视图，供后续剪枝、轨迹分析与实验决策使用。

核心实现只依赖 4 个文件，职责各占一层：

文件	职责	关键符号
src/param_importance_nlp/core/sufficient_statistics.py	流式、可合并的梯度充分统计量	S1/S2、G1/G2/N1/N2
src/param_importance_nlp/core/estimators.py	纯张量估计核与公开 score 包装	$\widehat{C}^U$ 、 $\widehat{C}^{\text{raw}}$
src/param_importance_nlp/core/accumulator.py	长期累计状态与不变量	signed / positive / negative / absolute
src/param_importance_nlp/runtime/training.py 的 OnlineImportanceTracker	把以上三者接入训练步与 checkpoint	micro_samples、clip_factor

推荐按这个顺序读：sufficient_statistics.py → estimators.py → accumulator.py → training.py 中的 OnlineImportanceTracker。前三者构成“估计数学”，第四个说明“如何在真实训练中产生、提交和恢复”。

1. 数学地基：core/sufficient_statistics.py

1.1 设计动机

估计器需要的不是每个 microbatch 的梯度本身，而是它的可合并汇总量：

$$S_1 = \sum_{m=1}^M g_m, \quad S_2 = \sum_{m=1}^M g_m^{\odot 2}$$

或加权版本：

$$G_1 = \sum_{m=1}^M w_m g_m, \quad G_2 = \sum_{m=1}^M w_m^2 g_m^{\odot 2}, \quad N_1 = \sum_{m=1}^M w_m, \quad N_2 = \sum_{m=1}^M w_m^2.$$

这种设计有三个直接收益：

- 1. **流式**。不需要保存 M 份梯度，每个参数张量只维护 S1/S2 两个同形状张量；
- 2. **可合并**。不同 shard（例如不同 GPU、不同恢复批次）的统计量只需加法归并，不需要重放原始梯度；
- 3. **跨 GPU 高效**。DDP 场景下只需 all-reduce 上述几个张量与标量，而不是 all-gather 每个参数梯度（见第 4 章）。

1.2 等权统计量 EqualSufficientStatistics

构造入口 from_samples（sufficient_statistics.py:87）用流式等价公式累计：

```
first = samples[0].to(dtype=accumulation_dtype)
s1_values = {name: torch.zeros_like(value) for name, value in first.items()}
s2_values = {name: torch.zeros_like(value) for name, value in first.items()}
for sample in samples:
    converted = sample.to(dtype=accumulation_dtype)
    for name, gradient in converted.items():
```

```
s1_values[name] = s1_values[name] + gradient
s2_values[name] = s2_values[name] + gradient.square()
```

这里有几个值得留意的点。所有梯度在进入统计量时统一转换到 `accumulation_dtype` (FP32 或 FP64)，避免 estimator 内部发生不可追溯的隐式精度变化。

S1/S2 都是交换求和，输入顺序不影响数学结果，所以 `merge` 可以放心地把两个 shard 的统计量直接相加 (`sufficient_statistics.py:122`)。但 `merge` 是 fail-closed 的 (`sufficient_statistics.py:125-135`)：dtype、`statistical_unit`、`sampling_design` 不一致时拒绝合并，防止把统计语义不同的 shard 静默拼在一起。比如一个 shard 按 microbatch 平均梯度、另一个按逐样本梯度，相加之后没有任何意义。

最后，`mean_gradient` (`sufficient_statistics.py:119`) 等于 S_1/M ，这是后续 raw 估计和优化器共用的平均梯度。

1.3 加权统计量 `WeightedSufficientStatistics`

语言模型场景中，每个 microbatch 的有效目标 token 数可能不同，此时不能简单等权。加权 U-statistic 需要四元组 G1/G2/N1/N2 (`sufficient_statistics.py:143`)。

`from_samples` 的累计逻辑 (`sufficient_statistics.py:191`)：

```
for sample, weight in zip(samples, numeric_weights, strict=True):
    converted = sample.to(dtype=accumulation_dtype)
    for name, gradient in converted.items():
        g1_values[name] = g1_values[name] + weight * gradient
        g2_values[name] = g2_values[name] + weight**2 * gradient.square()
```

注意 G2 累计的是 `weight**2 * gradient.square()`，而不是 `(weight * gradient).square()`。两者数学等价，但这里显式写出权重平方，与数学规格的 $\sum_m w_m^2 g_m^2$ 逐字对应，能降低实现歧义。

两个关键属性：

```
@property
def denominator(self) -> float:
    return self.n1**2 - self.n2

@property
def mean_gradient(self) -> TensorMap:
    return self.g1 / self.n1
```

分母 $N_1^2 - N_2 = \sum_{m \neq n} w_m w_n$ ，当 $M = 1$ 或权重全部集中在一个 microbatch 时趋于零，估计核会拒绝计算（见 2.2 节）。均值 G_1/N_1 是加权平均梯度，与优化器使用的全局平均梯度语义一致。

2. 估计核：`core/estimators.py`

2.1 设计原则：core 与 score 分离

文件头注释写得很清楚：所有 kernel 只返回“未乘学习率、未裁剪”的 `core`。学习率和裁剪属于训练 step 的尺度合同，统一由 `EstimatorResult.from_core` 应用。

这样分层的原因：

- kernel 是纯数学，可以在固定状态 fixture 中独立验证（FP64 oracle 对照）；
- 学习率是参数组级动态量，属于运行时事实，不该混进统计核；
- 裁剪因子来自同批随机梯度时，它本身是随机量，乘进 score 后会改变无偏性声明（见 2.3 节），所以必须在边界处显式记录 `clip_source`。

2.2 四个 kernel 逐个注释

raw: 同批平均梯度平方（对照）

```
def raw_importance(mean_gradient: TensorMap) -> TensorMap:
    return _square(mean_gradient)
```

公式: $\widehat{C}^{\text{raw}} = \eta_t \bar{g}_t^{\odot 2}$ 。

实现最简单，不需要额外前向/反向。但同一个 batch 同时用于更新方向和贡献评价，估计量会保留协方差偏差: $\mathbb{E}[\bar{g}^2] = \mu^2 + \sigma^2/M$ ，也就是正偏差来自随机梯度方差。因此它被定位为“关键对照”，不是 U-statistic 的别名。

double-sample: 两个独立样本的梯度乘积

```
def double_sample_importance(mean_gradient_a, mean_gradient_b):
    result = mean_gradient_a * mean_gradient_b
    ...
```

公式: $\widehat{\Omega}_D = \eta_t \bar{X}_A \bar{Y}_{B'}$ 。

两个独立 batch 的均值乘积在各自无偏条件下给出 $\mathbb{E}[\widehat{\Omega}_D] = \eta_t \mu_X \mu_Y$ ，边界很直观。代价是固定总样本预算时要把样本拆成两半，方差增大（数学规格 9.2 节给出了显式方差公式）。在线实现中，microbatch 按 $(\text{rank} + \text{local_index}) \% 2$ 稳定地切成两半（见 4.3 节），保证所有 rank 得到完全相同的 score。

equal-U: 等权去对角 U-statistic

```
def equal_u_importance(statistics: EqualSufficientStatistics) -> TensorMap:
    if statistics.count < 2:
        raise CoreContractError("等权 U-statistic 至少需要两个统计单元")
    denominator = statistics.count * (statistics.count - 1)
    result = (statistics.s1 * statistics.s1 - statistics.s2) / denominator
    ...
```

公式:

$$\widehat{C}_{k,t}^U = \eta_{g(k),t} \frac{S_{1,k,t}^2 - S_{2,k,t}}{M(M-1)} = \eta_{g(k),t} \frac{\sum_{m \neq n} g_{m,k,t} g_{n,k,t}}{M(M-1)}.$$

代数恒等式 $(S_1)^2 - S_2 = \sum_{m \neq n} g_m g_n$ 删掉了“自己乘自己”的对角项，从而消除 raw 的正偏差。 $M = 1$ 时无法删除对角项，必须抛异常。单步输出可能为负：即使目标 $\eta \mu^2 \geq 0$ ，无偏估计器的有限样本波动也允许出现负值。kernel 内绝不 `clamp_min(0)`，一旦截断就会重新引入正偏差。

weighted-U: 不等权去对角（正式主公式）

```
def weighted_u_importance(
    statistics: WeightedSufficientStatistics,
    *,
    require_unbiasedness_assumptions: bool = False,
) -> TensorMap:
    if statistics.count < 2:
        raise CoreContractError(...)
    denominator = statistics.denominator
    if not math.isfinite(denominator) or denominator <= 0:
        raise CoreContractError("加权 U-statistic 分母 N1**2-N2 必须为正")
```

```

if require_unbiasedness_assumptions and not (
    statistics.weights_exogenous and statistics.common_mean_assumption
):
    raise CoreContractError("未满足加权 U 的外生权重与共同均值声明")
result = (statistics.g1 * statistics.g1 - statistics.g2) / denominator
...

```

公式：

$$\widehat{\mu}_{U,w}^2 = \frac{(\sum_m w_m g_m)^2 - \sum_m w_m^2 g_m^2}{1 - \sum_m w_m^2}.$$

(代码用 $G_1^2 - G_2$ 与 $N_1^2 - N_2$ 表示，等权时退化为普通 U。)

几个行为要点：

- `require_unbiasedness_assumptions=True` 时，权重非外生或共同均值假设未声明都会直接失败，相当于把“声明防线”落到了数值层；
- 关闭该开关仍可计算描述性 plug-in 数值，但上层不得自动生成无偏声明；
- 分母必须为正：等权时对应 $M \geq 2$ ，加权时对应至少两个非零权重统计单元。

cross-U：一般交叉 U (X≠Y 场景)

```

def cross_u_importance(x_samples, y_samples, *, x_weights=None, y_weights=None,
                       exclude_matching_pairs=True):
    ...
    if not exclude_matching_pairs:
        return (sx / sum(wx)) * (sy / sum(wy))
    ...
    result = (sx * sy - diagonal) / denominator

```

公式（删除同索引对角项时）：

$$\frac{(\sum_m w_m^X X_m)(\sum_m w_m^Y Y_m) - \sum_m w_m^X w_m^Y X_m Y_m}{(\sum_m w_m^X)(\sum_m w_m^Y) - \sum_m w_m^X w_m^Y}.$$

该核估计 $\mathbb{E}[X]\mathbb{E}[Y]$ ，允许同一观测内部的 X/Y 相关，因此能覆盖“起点梯度与路径梯度来自同一统计单元”的一般路径场景。若两组样本来自彼此独立的 sampling stream，设 `exclude_matching_pairs=False`，就退化为两个加权均值的乘积（即 double-sample）。当前在线训练只使用前四个核，cross-U 主要服务于固定状态的对照验证。

2.3 global_clip_factor

```

squared_norm = sum(
    float(value.detach().to(torch.float64).square()).sum().item()
    for value in mean_gradient.values()
)
norm = math.sqrt(squared_norm)
return min(1.0, max_norm / (norm + epsilon))

```

计算在 FP64 下完成，避免 FP32 累加平方和时精度损失。返回 `min(1, G_max/(||g||+ε))`，只计算因子、不修改输入张量。`epsilon` 防止零梯度时除零，同时使因子恒有界于 [0,1]。

2.4 估计器一览表

公开名称	公式（未乘学习率）	目标	严格无偏条件
local_gradient_space_importance_u	$(S_1^2 - S_2)/(M(M - 1))$	$\eta\mu^2$	固定状态、未裁剪、i.i.d. 统计单元
local_gradient_space_importance_u_weighted	$(G_1^2 - G_2)/(N_1^2 - N_2)$	$\eta\mu^2$	上述条件 + 权重外生 + 共同均值
double_sample_gradient_importance	$\bar{g}_A\bar{g}_{B'}$	$\eta\mu_X\mu_Y$	两个独立 batch
raw_same_batch_gradient_importance	$\bar{g}^2$	-	无（保留正偏差作为对照）
*_clipped 系列	同核心 × 同批 clip 因子	在线 plug-in 分数	无， clip_source=same_batch_mean_gradient

3. 长期累计：core/accumulator.py

3.1 内部状态布局

ImportanceAccumulator (accumulator.py:30) 持有多组与参数同形状的 TensorMap：

分组	状态	含义
重要性四视图	_positive / _negative	正部与负部质量累计
派生态	_raw / _raw_clipped	raw 对照与同批 clip 的 plug-in raw
数据更新	_data_movement / _data_displacement	数据驱动更新的绝对路径与有符号位移
完整更新	_total_movement / _total_displacement	含 weight decay 的实际更新
权重衰减	_weight_decay_movement / _weight_decay_displacement	仅 weight decay 分量
端点	_magnitude / _initial_parameters / _last_parameters	幅值基线与首尾参数

公开视图由基础量派生，而不是独立浮点累计：

```
@property
def signed(self) -> TensorMap:
    return self._positive - self._negative

@property
def absolute(self) -> TensorMap:
    return self._positive + self._negative
```

这样四个视图永远满足代数恒等式：

$$\Omega^{\text{signed}} = \Omega^+ - \Omega^-, \quad \Omega^{\text{abs}} = \Omega^+ + \Omega^-.$$

3.2 add_step：原子式提交

add_step (accumulator.py:132) 先校验全部候选输入，再原地提交；任一输入非有限或坐标不一致时，不会产生部分累计。核心累计逻辑：

```
for name, value in converted.items():
    self._positive[name].add_(value.clamp_min(0))
```

```

        self._negative[name].add_((-value).clamp_min(0))
    if converted_raw is not None:
        for name, value in converted_raw.items():
            self._raw[name].add_(value)
    ...
    if converted_data is not None:
        for name, value in converted_data.items():
            self._data_movement[name].add_(value.abs())
            self._data_displacement[name].add_(value)

```

几个行为要点：

- 正负分解发生在累计层，而不是估计层。单步 score 可能为负，但累计器只保存非负的正部/负部质量，保证 `signed = positive - negative_mass` 逐坐标成立；
- raw 系列强制非负 (`accumulator.py:189-199`)。raw 是平方量，出现负值说明上游有 bug，直接抛 `CoreContractError`；
- movement 与 displacement 要分清。movement 累计每一步的 $|\delta|$ （路径长度），displacement 累计有符号 δ （净位移），`net_*_movement` 是净位移的绝对值。这样“来回震荡”和“单向移动”就能分开观察；
- `current_parameters` 只刷新幅值与最后端点，不参与正负质量累计。

3.3 validate_invariants

`validate_invariants` (`accumulator.py:252`) 逐坐标检查：

1. positive / negative mass 非负；
2. `signed == positive - negative_mass`；
3. `absolute == positive + negative_mass`；
4. 首尾端点差 == 累计总位移（仅在初始参数时）。

端点一致性用自适应容差处理 (`accumulator.py:268-274`)：首尾参数直接相减与逐 step delta 累加的浮点运算顺序不同，FP32 下不能要求逐位相等，因此按 `eps * max(1, steps) * 4` 放宽。容差只用于一致性报错，不会生成或修饰任何公开统计量，这是“校验不影响数值”的干净边界。

3.4 区间增量与断点恢复

- `delta_since(previous)` (`accumulator.py:284`) 返回两个 commit 边界之间的 signed / absolute / raw / movement 增量，是 Stage 4/5 阶段轨迹分析的原料；
- `state_dict()` (`accumulator.py:304`) 只输出 primitive 与 TensorMap，不使用 pickle 对象图，保证 checkpoint 可跨环境恢复；
- `load_state_dict()` (`accumulator.py:328`) 严格恢复，并对 0.3.x 的 v1 状态做无损可知迁移。v1 缺失的 clipped-raw、weight-decay 分解与参数端点无法反推，迁移时显式置零并保持 `has_initial_parameters=false`，不伪造历史。

3.5 数学累计公式表

视图	定义	用途
signed	$\sum_t \widehat{C}_{k,t}$	净损失作用，允许正负抵消
positive	$\sum_t [\widehat{C}_{k,t}]_+$	非负分布与剪枝排序
negative_mass	$\sum_t [-\widehat{C}_{k,t}]_+$	负贡献质量
absolute	$\sum_t \widehat{C}_{k,t}$	总活动强度
data / total / weight-decay movement	$\sum_t \delta_t$	路径长度
net movement	$\sum_t \delta_t$	净位移

4. 在线集成：runtime/training.py 中的 OnlineImportanceTracker

4.1 在训练引擎中的位置

`TrainingEngine._run_attempt` (`training.py:1436`) 的单步时序：

```
model.train() → optimizer.zero_grad()
→ _collect_micro_gradients(microbatches)      # 每个 microbatch 本地平均梯度
→ _global_mean_gradient(...)                  # 得到 micro_samples 与全局均值
→ GradientAttempt.capture().check_finite()     # 非有限则 skip
→ attempt.clip(max_grad_norm)                 # 得到 clip_factor
→ tracker.stage_distributed(micro_samples, weights, lrs, clip_factor)
→ bridge.step()                              # 真正执行 AdamW 更新
→ tracker.commit(main, raw_unclipped, raw_clipped, outcome)
→ scheduler.step() / checkpoint 发布
```

几个关键位置：

- `training.py:1478`：skip 时调用 `tracker.record_skip()`，不伪造零贡献；
- `training.py:1524`：`stage_distributed` 在 optimizer step 之前调用，只产生 score、不修改累计器；
- `training.py:1545`：`commit` 在 bridge step 之后调用，把 score 与真实更新 delta 一起原子提交。

4.2 构造

```
template = TensorMap(
    {name: registry.parameter(name).detach() for name in registry.eligible_names},
    registry=registry,
)
self.accumulator = ImportanceAccumulator(
    template, accumulation_dtype=_dtype_from_name(spec.accumulation_dtype)
)
self.accumulator.set_initial_parameters(template)
```

统计范围由 `registry.eligible_names` 决定，避免把冻结或不可训练参数混入。训练起点即被记为初始参数端点，供第 3 章的首尾位移校验使用。

4.3 `stage_distributed`：全局归约 + 估计

这是在线估计的心脏 (`training.py:695`)，分四步。

第一步：本地加权统计量

```
converted = [sample.to(dtype=dtype) for sample in micro_gradients]
g1 = TensorMap.zeros_like(converted[0], dtype=dtype)
g2 = TensorMap.zeros_like(converted[0], dtype=dtype)
for sample, weight in zip(converted, weights, strict=True):
    g1 = g1 + sample * float(weight)
    g2 = g2 + sample.map(torch.square) * float(weight) ** 2
```

对应数学规格的 $G_1 = \sum w_m g_m$ 、 $G_2 = \sum w_m^2 g_m^2$ 。

第二步：只归约统计量，不归约梯度

```
reduced = reducer.sum_tensors({
    **{f"g1:{name}": value for name, value in g1.items()},
    **{f"g2:{name}": value for name, value in g2.items()},
    **scalars,      # __n1__ = sum(weights), __n2__ = sum(weights**2)
})
```

这是 DDP 语义的关键：U 只需要全局 G1/G2/N1/N2/count，无需 all-gather 每个参数梯度。所有 rank 因此得到完全相同的 estimator score，通信量也从参数数量级降到统计量数量级。

第三步：构造统计量与 raw 对照

```
global_statistics = WeightedSufficientStatistics(
    count=reducer.sum_int(len(converted)),
    g1=..., g2=..., n1=..., n2=...,
    accumulation_dtype=dtype,
    statistical_unit="microbatch_mean_gradient",
    weight_unit="effective_loss_units",
    sampling_design="ordered_disjoint_microbatches",
    weights_exogenous=self.spec.weights_exogenous,
    common_mean_assumption=self.spec.common_mean_assumption,
)
```

声明字段在这里一次性写入统计量对象，之后 `EstimatorResult.from_weighted_u` 会自动决定无偏性声明。

```
clip_source = "none" if clip_factor == 1.0 else "same_batch_mean_gradient"
raw_unclipped = EstimatorResult.from_core("raw_same_batch_gradient_importance", ...)
raw_clipped = EstimatorResult.from_core("raw_same_batch_gradient_importance_clipped", ...)
```

第四步：按配置选择主估计器

- `estimator_name == "raw"`：直接返回 raw 系列；
- `estimator_name == "u"`（默认主指标）：调用 `EstimatorResult.from_weighted_u(...)`，得到 `local_gradient_space_importance_u_weighted` 或 `*_clipped`；
- 其余走 double-sample：把本地 microbatch 按 $(rank + local_micro_index) \% 2$ 稳定分成两半，分别 all-reduce 加权梯度与权重，再计算两个全局均值之积。

4.4 commit：把分数与真实更新绑定

```
def commit(self, main, raw_unclipped, raw_clipped, outcome):
    data = TensorMap(outcome.data_delta, registry=self.registry)
    total = TensorMap(outcome.total_delta, registry=self.registry)
    ...
    self.accumulator.add_step(
        main.score,
        raw=raw_unclipped.score,
        raw_clipped=raw_clipped.score,
        data_update=data,
        total_update=total,
        weight_decay_update=...,
        current_parameters=parameters,
    )
```

- `main.score` 是已经乘过学习率和 clip 因子的公开分数，正负分解发生在 `add_step` 内；
- 同时记录数据驱动更新（不含 weight decay）与完整更新，使“权重衰减贡献”能从 total 中分离出来单独分析；
- skip 的 attempt 只走 `record_skip`，绝不传入零贡献伪装成功步，保证 `successful_steps` 语义干净。

4.5 checkpoint 与 trajectory

- `_checkpoint_state()` (training.py:1250) 把 `tracker.accumulator.state_dict()` 写入 checkpoint，与模型/optimizer/scheduler/RNG/数据游标同构存储；
- resume 时调用 `accumulator.load_state_dict(importance)` (training.py:1419) 严格恢复；

- 每次保存 checkpoint 时生成 `ImportanceTrajectoryPoint(global_step, checkpoint_id, snapshot)` (`training.py:1278-1283`), 构成可审计的重要性轨迹;
- 若 checkpoint 发布失败, 内存中的 trajectory point 会被回滚 (`training.py:1290-1293`), 不产生孤儿记录。

附录 A：符号表

符号	含义
M	一个 optimizer step 内的全局 microbatch 数
g_m	第 m 个 microbatch 的未同步、未裁剪平均梯度
w_m	第 m 个 microbatch 的有效统计单元权重 (有效 token 数)
S_1, S_2	等权累计 $\sum g_m$ 、 $\sum g_m^2$
G_1, G_2, N_1, N_2	加权累计 $\sum wg$ 、 $\sum w^2 g^2$ 、 $\sum w$ 、 $\sum w^2$
$\eta_{g(k),t}$	参数 k 所属 optimizer group 在第 t 步的学习率
s_t	同批全局裁剪因子 $\min(1, G_{\max}/(\bar{g} + \varepsilon))$
$\widehat{C}_{k,t}$	第 t 步、参数 k 的单步重要性贡献
$\Omega_k^{\text{signed}/+/-/\text{abs}}$	跨步累计的四个视图